

Deep Learning: Image Sentence Pair Classification

Popescu Ioana-Livia
Group 507, Artificial Intelligence
University of Bucharest
ioana.popescu1@s.unibuc.ro

January 19, 2025

1 Introduction

The provided task is to perform binary classification of image + sentence pairs. The sentences are interpreted as potential captions - they could either match the image (label 1) or not (label 0). Among the rules, the use of additional data, pre-defined models or pre-trained models has been prohibited. The following document presents my approach, ranging from simpler to more complex models, documenting both successful and unsuccessful experiments.

2 Dataset

2.1 Provided data

The provided corpus consists of 10000 (train), 3000 (validation) and 2000 (test) image-sentence pairs. For the train and validation sets, the samples are evenly distributed in terms of label distribution.

2.2 Updated test set captions

The test set only contains 15 unique values for captions out of which a few contain grammatical errors such as "mterioal", "fitngesr", "smafml", "instqrumemnt".

In order to address this limitation, with respect to the prohibited use of additional data, a mapping between the words from the test set and their closer match based on the Damerau-Levenshtein distance (considering possible substitutions, deletions, insertions, transpositions) has been applied and resulted in an updated test set for captions. While some of the grammatical errors were potentially correctly handled ("smafml" → "small"), others were not ("instqrumemnt") as a close enough word could not be found in the train or validation set. This is expected as an upper threshold of 2 has been enforced for potential word replacements based on the Damerau-Levenshtein distance. The threshold has been set in an effort to avoid losing important words for the caption by performing too many replacements or replacements with words too different from the initial ones.

2.3 Tokenization

Taking into consideration that some words of the test captions are not present in the train and validation set and many others might be misspelled, a custom tokenizer based on Byte-Pair Encoding (BPE) has been implemented and trained on the train, validation and test captions. As we don't have a lot of data, also considering the words per caption (Table 1) and characters per caption (Table 2), the maximum size for the vocabulary has been set to 5000. The subword-based tokenization method relies on creating an initial set of tokens (in this case, utf-8 has been chosen such that initial value between 0-255 will be assigned to each character) ; afterwards, the most frequent pairs of tokens are merged into new ones until the maximum size of vocabulary is reached *size_of_index.to.associated.byte.value.vocab* or the frequency of the pair is not greater or equal than a set threshold *lower_bound.fq.pair*. The majority of experiments have been conducted with *lower_bound.fq.pair* = 10, leading to a vocabulary size of 2469. Experiments have been conducted with *lower_bound.fq.pair* = 0 (vocabulary size of 5000),

$lower_bound_fq_pair = 3$ (vocabulary size of 4713), $lower_bound_fq_pair = 20$ (vocabulary size 1808). The threshold of 20 proved to be too high, while a threshold of 3 or lower did not result in a significant improvement of results.

Table 1: Words per caption

Dataset	<i>Average</i>	<i>Min</i>	<i>Max</i>
Train	10.2652	7	46
Validation	10.204(3)	7	43
Test	16.808	1	50
Train + Validation + Test	11.1254	1	50

Table 2: Characters per caption

Dataset	<i>Average</i>	<i>Min</i>	<i>Max</i>
Train	52.7765	27	250
Validation	52.481	27	220
Test	93.35	14	278
Train + Validation + Test	58.1272	14	278

2.4 Image Preprocessing

The provided images are 100 x 100 pixels, colored - with a few exceptions which were found to be grayscale. The possible mismatch in size and number of channels is handled in the defined Datasets (torch.utils.data) classes, ensuring a size of 100 x 100 pixels, 3 channels and an additional normalization. Due to limitations of time and computational resources, experiments with augmented versions of the dataset obtained by modifying the images (brightness, contrast, blur, flip, rotation, sharpening etc.) have not been conducted, but could be considered for future work.

2.5 Final datasets

The final datasets are implemented via Dataset (torch.utils.data) classes and contained the tokenized captions, preprocessed images, labels (for train and validation) and id (to later identify predictions). Another important attribute of the classes is *remove_negative_matches* which filters out the entries that are labeled with 0 (not considered an image-sentence match). This setting has been implemented to be able to further support the logic of the chosen loss function presented below.

3 Approach

3.1 Loss function

Possibly one of the most important aspects of my approach has been introducing a loss function based on CLIP (Contrastive Language Image Pretraining) - this brought the simpler approaches above the baseline. As the goal is to maximize the cosine similarity between the image embedding and the corresponding text embedding, for training only the entries with label 1 have been kept.

3.2 Models

My approach consisted in starting with simpler architectures and then moving onto implementing adaptations of more complex models from scratch. In terms of text, considering the limited amount of data, I explored multiple simpler approaches based on 1D convolutions which ended up performing well when combined with more complex image models such as adaptations of ResNet18 and ResNet34.

I chose to use ResNet-based models with consideration to its original state-of-the-art performance for image classification tasks and current computational resources. The best results have been obtained using the ResNet18-based approach. Another key aspect has been introducing dropout - for both text and image models. Although unusual for ResNet, introducing dropout made a positive impact on results, perhaps due to the limited size of the dataset and the risk of overfitting. Experiments with different dropout rates, dropout placed in text/image models at different layers have been conducted and are described below. In the conducted experiments the models have been trained for 100 epochs, with consideration to computational limitations.

Approach 1 : The first approach to be presented is based on simpler models - without introducing any dropout. The image architecture presents 3 convolutional layers (with the below settings):

- in_channels = 3, out_channels = 16, kernel_size = 3, stride = 2
- in_channels = 16, out_channels = 64, kernel_size = 3, stride = 2
- in_channels = 64, out_channels = 128, kernel_size = 3, stride = 2

each followed by a ReLU activation function, with an additional adaptive max pool and flatten layer at the end. For the text model (which will be further referenced at text_model_1, an embedding of *num_embeddings* = size of the vocabulary of the utilized tokenizer is applied first and *embedding_dim* = 128, followed by 2 1D convolutional layers with the following settings:

- in_channels = 128, out_channels = 64, kernel_size = 3, stride = 1, padding = 1
- in_channels = 64, out_channels = 128, kernel_size = 3, stride = 1, padding = 1

followed by a ReLU activation function, adaptive max pool and flatten layer.

For the top submission based on this approach, trained only on train, for 6 epochs, using the AdamW optimizer, lr = 0.001, utilizing the tokenizer with a set threshold of 10, obtained 68.4% accuracy on validation, a public score of 0.59250 and a private score of 0.56375. The confusion matrix based on the validation set is presented below:

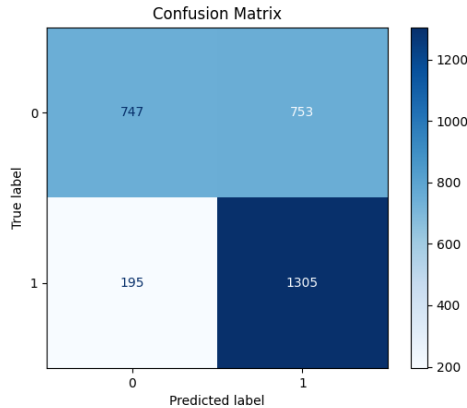


Figure 1: Confusion matrix for approach 1

Approach 2 : The second approach is based on text_model_2 and an a custom implementation of ResNet18 where a dropout rate can be added at residual block level. The text_model_2 has an identical first embedding layer as text_model_1, followed by a dropout layer (for which experiments with values in the range of 0-0.5 have been conducted), 3 1d convolutional layers :

- in_channels = 128, out_channels = 128, kernel_size = 3, padding = 1
- in_channels = 128, out_channels = 64, kernel_size = 5, padding = 2
- in_channels = 64, out_channels = 128, kernel_size = 3, padding = 1

each followed by a normalization layer and a max pooling layer, the latter having an adaptive max pooling layer, followed by the final flatten layer.

The top submission is based on this approach, trained on both train and validation for 41 epochs, with a dropout (for image model) of 0.25 and a dropout (for text model) of 0.15, learning rate of 0.001, AdamW optimizer and the tokenizer with the threshold set to 10 has been utilized. The obtained private score 0.69562 is and the obtained public score is 0.69750. For the same settings, but trained on train, the top validation accuracy obtained is 69.76% and the confusion matrix can be seen below:

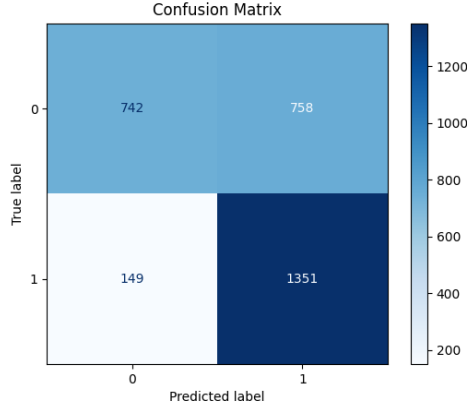


Figure 2: Confusion matrix for approach 2

Approach 3 : The third approach involves an adaptation of ResNet34 for the image model and text_model_3 which is similar to text_model_2, however dropout is not applied only after each ReLU activation function that follows the 1d convolutional layers mentioned above and the normalization layers, alongside the first two max pooling layer have been removed.

The top submission based on this approached, trained only on train data for 62 epochs, utilizing the tokenizer with the threshold set to 10, a dropout rate of 0.3 for the text model, learning rate of 0.0001 obtained a validation accuracy of 68.4%, private score of 0.62250 and public score of 0.66500.

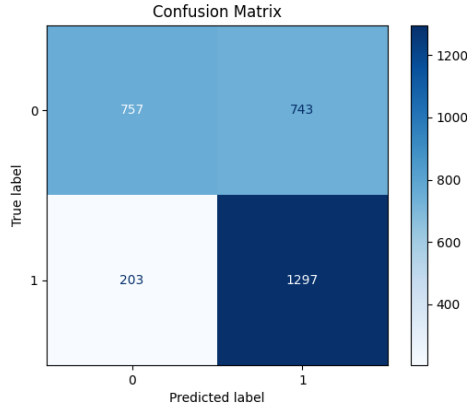


Figure 3: Confusion matrix for approach 3

However, another approach based on the ResNet34-based model and text_model_2 outperformed approach 3, with a learning rate of 0.001, trained on train for 91 epochs, with a dropout of 0.25 for the image model, utilizing the tokenizer with the set threshold to 10, has been able to obtain a public score of 0.62500 and a private score of 0.65187.

4 Results

Model_Image	Model_Text	LR	Dropout_Img	Dropout_Txt
ResNet18 (trained on train+val)	text_model_2	0.001	0.25	0.15
ResNet18	text_model_2	0.001	0.25	0.2
ResNet18	text_model_2	0.001	0.25	-
ResNet34	text_model_2	0.001	0.25	-
ResNet34	text_model_3	0.0001	-	0.3

Table 3: Configurations for top submissions

Epochs	Validation Acc	Public score	Private score
41	-	0.69750	0.69562
63	64.75%	0.69750	0.65062
46	63.50%	0.66250	0.66
91	68.47%	0.62500	0.65187
62	68.4%	0.66500	0.62250

Table 4: Scores for top submissions